

BİL107U-İLERİ PROGRAMLAMA I

Ünite 1: Algoritma ve Programlamaya Giriş

Giriş

Programların, karşılaşılan bir problemi çözme amacıyla, hayatımızı kolaylaştırmak için hazırlanmış işlemler olduğunu söyleyebiliriz. Bu yapılan işlemleri, bir yemeğin yapılış aşamaları yani tarifi gibi de düşünebiliriz. Tarife uygun yapılmayan bir yemeğin sonucunu bir hayal edelim. Çevremizde her gün karşılaştığımız ama belki de fark etmediğimiz olaylar da aslında bir problemin çözümüdür. Yanından geçtiğimiz bir inşaatı düşünün; inşaatı tamamlamak için tüm yapılan işlerin sırasıyla yapılması gerekmektedir. Temel kazılmadan inşaatı başlanılamayacağı gibi duvar örmeden de sıra yapılamaz. Program yazmak için de belirli aşamaların geçilmesi gerekmektedir. İlk olarak ihtiyaç belirlenmelidir. Sonra bu ihtiyacı çözmek için konu hakkında gerekli olan tüm bilgiler öğrenilmelidir. Soru sormak, konu hakkındaki bilgiyi arttıracak gibi çözüm hakkında fikir sahibi olunmasını da sağlayacaktır. Ayrıca karşılaştığımız farklı problemler için çözüm yollarını ve nasıl bir yöntem uygulanacağını düşünmek de programlama becerimizi geliştirecektir.

Algoritma

Algoritma Nedir?

Algoritma dediğimiz kavram, belirli bir problemi çözmek ya da bir hedefe ulaşmak amacıyla izlenmesi gereken yola verilen isimdir. Bu yolda belirlenen adımlar da mantıksal olarak birbirini takip etmeli, hepsi açık, düzenli ve sıralı bir şekilde ifade edilmelidir. Algoritmalarda dikkat edilmesi gereken hususları ise şu şekilde ifade edebiliriz:

- İlgili adım, herkesin anlayabileceği gibi net bir şekilde ifade edilmelidir.
- Probleme uygun olarak algoritma adımları arasındaki işlem sırasına dikkat edilmelidir.
- Tüm olası durumları içerecek kadar genel olmalıdır.
- Başlangıç-bitiş adımlarını içermeli, sonlu sayıda işlem adımına sahip olmalı ve belirli bir sürede bitmelidir.

Peki algoritma neden önemlidir?

- Problem çözümü herkes tarafından anlaşılabilirdiği için takım çalışmasını da kolaylaştırır.
- Projeyi tasarlayan haricinde bir başka kişi tarafından da kodu yazılabilir.
- Kodlama öncesi kontrollerin yapılarak mantıksal hataların tespit edilmesini kolaylaştırır.

Problem Çözme

Problem, gündelik hayatta karşılaştığımız bir sorun, bir otomasyon sistemi ya da işlemin bilgisayar ile çözülebilmesi fikrinin ortaya çıkmasıdır. İnsan gücü ile çok uzun zaman alacak ya da çözülemeyecek sorunların bilgisayar vasıtası ile çözülmek istenmesi ile bilgisayar problemleri kavramı oluşmuştur.

Belirlenen problemin bilgisayar ortamında çözüm aşamaları ise şu şekilde sıralanabilir:

1. **Problemi Tanımlama (İnceleme, Analiz):** Algoritmanın amacı problemi çözmektir. Çözülecek problem ne kadar iyi anlaşılır ise algoritmayı geliştirmek o kadar kolay olacaktır.
2. **Girdi ve Çıktıların Belirlenmesi:** Problemi iyi tanımlayabilmek için başlangıç ve bitiş noktaları belirlenmiş, sistemin girdi ve çıktı parametrelerinin ortaya konulmuş olması beklenmektedir.
3. **Alternatif Çözüm Yolları:** Bir problemin birden fazla çözüm yolu olduğu unutulmamalıdır. Zamana ve insan kaynağına göre en uygun çözüm yolu belirlenmelidir. Algoritma, seçilen çözümü adımlara ayırıştırıp birbirleri ile ilişkilerinin düzgün biçimde kurulmasını sağlayacak şekilde dizayn edilmelidir.
4. **Çözümün Sınanması ve İyileştirilmesi:** Hazırlanan algoritmanın kodlamadan önce nasıl çalışacağını adım adım incelenmesi, varsa eksik ve hataların giderilmesi ve gerekli görülen değişikliklerin yapılması gerekmektedir. Bu işlemler sırasında işlemci ve bellek kullanımı konusunda görülen iyileştirme imkânları da mutlaka göz önünde bulundurulmalıdır.
5. **Algoritmanın Kodlanması:** Problemin çözümü için geliştirilen algoritma, bilgisayar üzerinde çalışabilmesi için ihtiyaca uygun olarak belirlenen bir programlama dili ile kodlanır.
6. **Kodun Sınanması ve İyileştirilmesi:** Yazılan kod da tıpkı algoritmada olduğu gibi sınanmalıdır. Bu seferki sınama, belirlenen girdiler için belirlenen çıktıların alınmasının kontrol edilmesi amacıyla kodun çalıştırılmasıdır. Bu aşamada da performans ve olası hataların düzeltilmesi beklenmektedir.

Akış Diyagramı

Akış diyagramları algoritmanın, görsel ve şekilsel olarak ifade edilmesi için kullanılır. Algoritmadan farklı olarak programın başlangıcından bitişine kadar olan tüm adımlar, geometrik şekillerle gösterilmektedir. Her bir geometrik şekil ve simge, bir iş veya komuta karşılık gelmektedir. Bu yüzden algoritmadaki gibi her adım için açıklama yapma zorunluluğu yoktur. Algoritmadaki adımlar arası geçişler, akış diyagramında oklar ile temsil edilmektedir. Akış diyagramında kullanılan şekillerin uluslararası geçerliliği olduğundan, herkes tarafından rahatça anlaşılabilir.

Akış Diyagramında Kullanılan Şekiller

Akış diyagramında kullanılan şekillerin içlerine değışkene atanan değerler, değışken isimleri, karşılaştırmalar ve işlemler yazılabilir ancak burada kullanılan ifadeler algoritmada kullanılanlar kadar serbest şekilde kullanılmamalıdır. Kullanılan ifadeler kısa, net ve anlaşılır olmalıdır. Akış diyagramlarında kullanılan şekillerin adı



ve açıklaması kitabınızın 7. sayfasındaki Tablo 1.2 de verilmiştir.

Akış Diyagramı Türleri

Akış diyagramları, kullanılan algoritmaların içerik ve biçimine göre genel olarak üç grupta sınıflandırılabilir.

Doğrusal (Sıralı)

Akış içerisinde giriş, hesaplama ve çıkış biçiminde sıralı bir olay kurgusu vardır. Her işlem bir diğerini takip ederken işlemler içinde koşul veya döngü gerçekleşmez. Örnek olarak,

- Girilen iki sayının çarpımını bulma,
- Çemberin çevre ve alanını bulma,
- Notları girilen öğrencilerin sınıf ortalamasını bulma

verilebilir.

Mantıksal (Koşullu)

Akışın yönünün bazı kural ve koşullara göre değiştiği akış şemalarıdır. Ayrıca koşulların durumuna göre bazı adımlar gerçekleşmez. Örnek olarak,

- Girilen iki sayıdan büyük olanı bulma,
- 3 sayının en küçüğünü bulma,
- Girilen bir sayının tek-çift olduğunu anlama

verilebilir.

Döngüsel (Tekrarlı)

Mantıksal akış şemasının özelleşmiş bir versiyonu olduğu söylenebilir. Koşula göre bazı adımların tekrarlandığı akış şemalarıdır. Örnek olarak,

- Girilen bir kelimeyi ekrana 100 defa yazdırma,
- 1'den N'ye kadar olan sayıların toplamını bulma,
- Girilen bir sayının faktöriyelini hesaplama

verilebilir.

Sözde Kod (Pseudo Code)

Problem çözümü için tasarlanan algoritmanın kodlanması gerekmektedir. Sözde Kod (PseudoCode), bu algoritmanın bir programlama diline dönüştürülmeden önceki bir ara formu olarak düşünülebilir. Bu ara form, algoritmanın yarı programlama dili yarı konuşma dili şeklinde ifade edilmesidir. Sözde kod şeklinde verilen bir algorithma, veri yapıları gibi detaylar göz ardı edilerek daha çok algoritmanın davranışı üzerine odaklanılır. Ayrıca sözde kod oluşturulurken herhangi bir söz dizimi kuralı yoktur. Böylece algoritmamız, daha fazla kişi tarafından anlaşılabilir ve sözde kodu alan bir yazılımcı da ihtiyaca göre seçilen programlama dilinde kolay bir şekilde kodlayabilecektir. Sözde kodun hazırlanmasının bir diğer faydası da, kodlama bilmeyen kişilerin de yapılacak işin akışını ve mantığını kavramalarında kolaylık sağlamasıdır. Burada dikkat edilmesi gereken en önemli nokta, metnin anlaşılır ve kolaylıkla bir programlama diline uyarlanabilir olmasıdır. Genel olarak

kullanılan sözde kod komutlarını şu şekilde listeleyebiliriz:

- Başla (begin): Programın başladığını gösterir.
- Bitir (end): Programın bittiğini gösterir.
- Oku (input, read, get, accept): Kullanıcıdan veri alır.
- Yaz (print, display, write): Kullanıcıya çıkış birimlerinden sonuç gösterir.
- Eğer ... İse ... (if ... then ... [if ... then]) ve Eğer ... ise ... Değilse Eğer ... (if ... then ... else): Koşullara göre akışa yön vermek için kullanılır.
- Döngü (sayaç= [Başlangıç değeri] TO [Hedef sayı değeri] STEP [Artış/azalış miktarı]) ... Döngü sonu (for ... endfor): Kod içerisinde, sayısı belli olan tekrarlı blokların oluşturulmasında kullanılır.
- Döngü (kontrol) ... Döngü sonu (while ... endwhile): Belli bir koşul sağlanana kadar yapılması gereken işlem bloklarında kullanılır.

Yazılım Geliştirme Süreci

Taleplerin Belirlenmesi

Yazılım geliştirmenin ilk başta ele alınması gereken ve en önemli adımıdır. İhtiyacın belirlenerek buna uygun çözümün geliştirilmesi için, odaklanılması gereken yerler doğru tespit edilmelidir. Yazılım geliştirme sürecinin ilk adımı olduğu ve bundan sonraki tüm adımları doğrudan etkilediği için, tüm ihtiyaçların net olarak anlaşılması gerekmektedir.

Planlama ve Analiz

Bu aşamada, ilk adımda ortaya çıkarılan modüller detaylı olarak analiz edilmeli, projenin hedefleri ve amaçları detaylandırılmalıdır. Çözümde elde edilen tüm çıktılar ve beklentilerin neler olduğu kesin olarak belirtilmelidir. Yine bu aşamada, projede zaman kaybına neden olacak önemli görülmeyen ve acil olmayan özellikler, sonraki fazlara aktarılabilir.

Tasarım

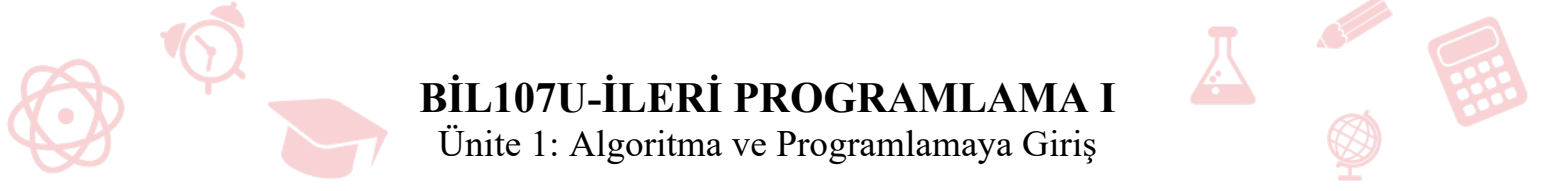
Bu aşamada ise hazırlanan projenin teknik kısımları ele alınmaktadır. Kullanıcı arayüzleri, veri tabanı şemaları, UML (Unified Modelling Language-Birleşik Modelleme Dili) diyagramları hazırlanır. Tasarımlar yapılırken yazılım standartlarına uygun şekilde dizayn edilmesi gerekmektedir.

Kodlama

Bu aşamada ise tasarım aşamasında oluşturulan tüm dokümanlar ve istekler yazılım geliştirme ekibine iletilmektedir. Yazılım geliştirme süreci de kod yazım standartlarına uyularak yazılım geliştirme ekibi tarafından gerçekleştirilir.

Test ve Entegrasyon

Geliştirilen modüller, paketler ve akışların, müşterinin taleplerine uygun şekilde hazırlanan senaryolara göre test ekibince test edildiği aşamadır.



BİL107U-İLERİ PROGRAMLAMA I

Ünite 1: Algoritma ve Programlamaya Giriş

Bakım

Her ne kadar son sistem test araçları kullanılsa da, birim ve sistem testleri başarılı bir şekilde geçse de, operasyon sırasında kullanıcılar hata alabilirler. Burada önemli olan alınan hatalar için hızlı bir şekilde ve kalıcı çözüm üretebilmektir. Bakım süreci, projenin büyüklüğüne göre şekillenmektedir. Operasyon kısmında artık sorun alınmadığı zaman bu süreç de sonlandırılarak, standart bakım sürecine geçilebilir.

Günlük Hayattan Algoritma Örnekleri

Günlük hayatta yaptığımız çoğu eylem de algoritmalara örnek olarak verilebilir. Bunların çoğunu pratik bir şekilde ve sürekli yaptığımız için, eylemdeki algoritma adımlarını fark etmeyiz. Yaptığımız en küçük bir işte bile, yine bir algoritmanın takip edildiğini göreceksiniz.

- Hava durumuna göre yanımıza şemsiye, şapka alma,
- Çay demleme,
- Asansörün 250 kg üzeri yük geldiğinde çalışmayı durdurması,
- Çamaşır yıkama ve asma,
- İşe gitmek için yapılan hazırlıklar

gibi örnekler verebiliriz.

Hava durumuna göre yanınıza şemsiye, şapka alma

1. Başla.
2. Hava yağmurlu veya kapalı mı?
3. Evet ise 10. adıma git.
4. Hayır ise 5. adıma git.
5. Hava çok güneşli mi?
6. Evet ise 8. adıma git.
7. Hayır ise 11. adıma git.
8. Şapkamı yanına al.
9. 11. adıma git.
10. Şemsiyeyi yanına al.
11. Bitir

